

eBook Ambientes de Dev. de Software:

GUIA PRÁTICO COM JAVASCRIPT, VUE E NODE.JS

A large, bold, dark gray 'JS' logo is positioned on the right side of the cover. The background is a bright yellow triangle that points towards the top right corner. The 'J' and 'S' are thick and modern in style.

01.

Objetivo Geral do Projeto

02.

Linguagem: JavaScript

03.

JavaScript com Node.js
e Framework Vue.js

04.

Versionamento com Git

05.

ESLint: Garantindo Qualidade
no Código

06.

Virtualizando Aplicações:
Docker

SUMÁRIO

01.

OBJETIVO GERAL

Este e-book tem como objetivo reunir, de forma completa e estruturada, todo o aprendizado adquirido na disciplina de Ambientes e Desenvolvimento de Software. Além disso, exploramos as tecnologias escolhidas para o projeto: JavaScript como linguagem principal, Vue como framework e Node.js como plataforma de desenvolvimento.

O conteúdo está organizado em seis partes, conforme apresentado no sumário. Essa divisão permite abordar os principais tópicos da disciplina, complementando a teoria com exemplos práticos para facilitar a compreensão e o aprendizado.

02. LINGUAGEM: JAVASCRIPT

JavaScript é uma linguagem de programação amplamente **utilizada para desenvolvimento Web**, permitindo a criação de interatividade e dinamismo em páginas da web. Originalmente desenvolvida para ser executada no lado do cliente (navegador), JavaScript também pode ser usado no lado do servidor, especialmente com a popularidade do Node.js. Além disso, JavaScript possui tipagem **dinâmica e fraca**, o que significa que as variáveis podem ser atribuídas a diferentes tipos de dados ao longo do tempo, e a verificação de tipos é feita em tempo de execução.

A large, bold, black 'JS' logo is centered on a yellow background. The letters are thick and have a slightly rounded, modern font style.

VARIÁVEIS: JAVASCRIPT

- **String:**

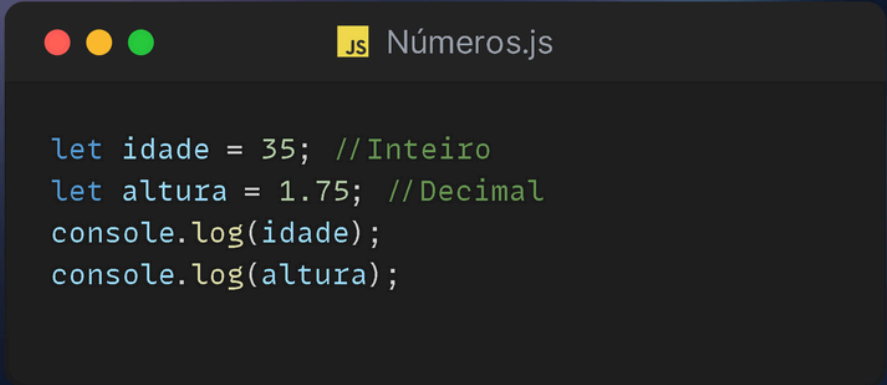
Uma sequência de caracteres usada para **representar texto**. Strings podem ser delimitadas por aspas simples, duplas ou crase.



```
let nome = "Miqueias"; //String
console.log(nome);
```

- **Números:**

Representa números, **tanto inteiros quanto decimais**. JavaScript não diferencia entre tipos de números, todos são do tipo *number*

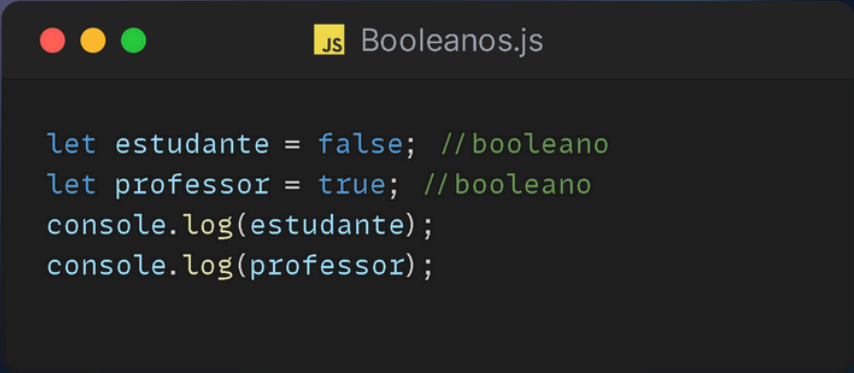


```
let idade = 35; //Inteiro
let altura = 1.75; //Decimal
console.log(idade);
console.log(altura);
```

VARIÁVEIS: JAVASCRIPT

- **Booleanos:**

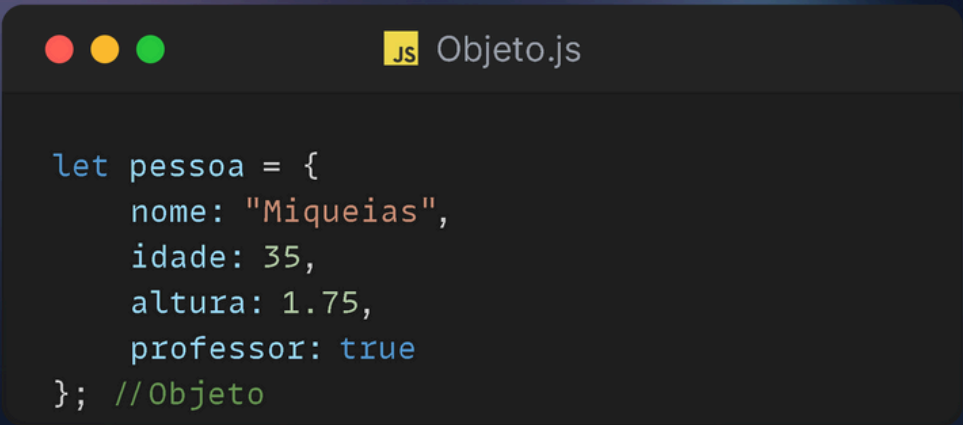
Um tipo de dado que pode ser **verdadeiro (true)** ou **falso (false)**. É usado em condições e expressões lógicas.



```
let estudante = false; //booleano
let professor = true; //booleano
console.log(estudante);
console.log(professor);
```

- **Objeto:**

Um tipo de dado que permite armazenar **coleções de dados e entidades mais complexas**. Objetos são compostos por pares chave-valor e podem conter outros objetos, arrays e funções.

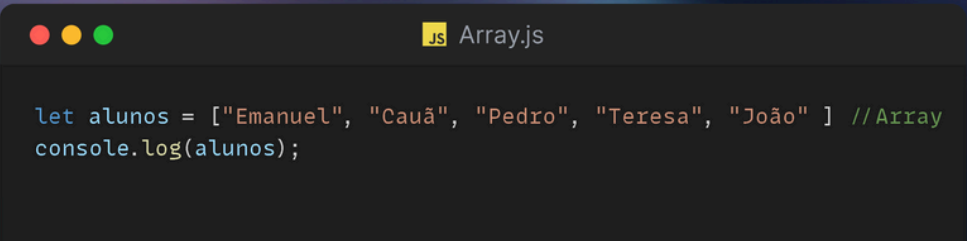


```
let pessoa = {
  nome: "Miqueias",
  idade: 35,
  altura: 1.75,
  professor: true
}; //Objeto
```

VARIÁVEIS: JAVASCRIPT

- **Array:**

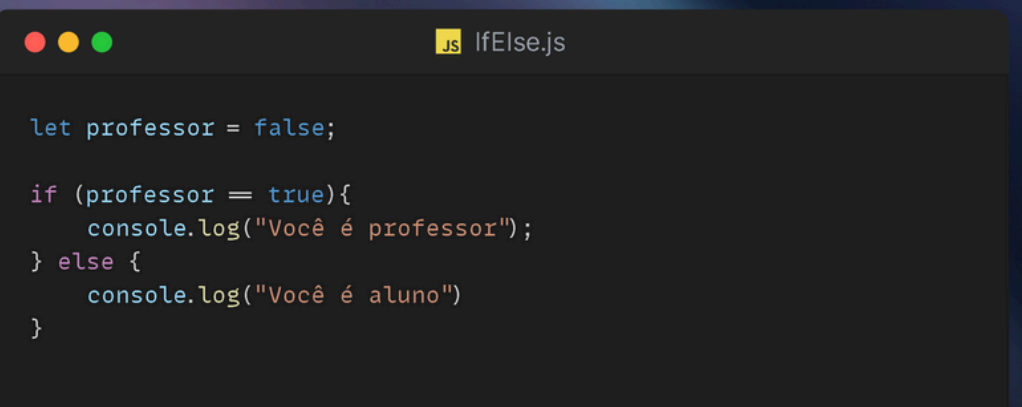
Uma estrutura de dados que **armazena uma lista ordenada de valores**. Arrays são objetos especiais que têm métodos e propriedades para manipulação de listas.



```
let alunos = ["Emanuel", "Cauã", "Pedro", "Teresa", "João"] //Array
console.log(alunos);
```

- **If e Else:**

A estrutura **if** em JavaScript permite executar um bloco de código se uma condição for verdadeira, avaliando uma expressão booleana. Se a condição do if não for atendida, o bloco else pode ser usado para executar um código alternativo. Você também pode usar **else if** para testar múltiplas condições em sequência. Essa estrutura de controle é fundamental para a lógica condicional em programas JavaScript.



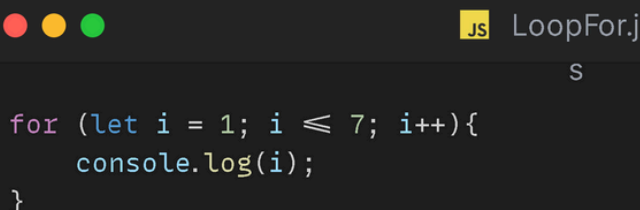
```
let professor = false;

if (professor = true){
  console.log("Você é professor");
} else {
  console.log("Você é aluno")
}
```

VARIÁVEIS: JAVASCRIPT

• Loop *For*:

O loop for em JavaScript **permite executar um bloco de código repetidamente, definindo uma inicialização, uma condição e uma atualização**. Ele é útil para iterar sobre sequências, como arrays, ou executar um código um número específico de vezes.

A screenshot of a code editor window titled "LoopFor.js". The editor has a dark background and shows a JavaScript for loop. The code is:

```
for (let i = 1; i ≤ 7; i++){  
  console.log(i);  
}
```

• Loop *While*:

O loop while em JavaScript **executa um bloco de código repetidamente enquanto uma condição especificada for verdadeira**. É útil quando o número de iterações não é conhecido de antemão e depende de uma condição que pode mudar durante a execução.

A screenshot of a code editor window titled "LoopWhile.js". The editor has a dark background and shows a JavaScript while loop. The code is:

```
let i = 1;  
  
while(i ≤ 7){  
  console.log(i);  
  i++;  
}
```


MOTIVOS PARA O USO DO JAVASCRIPT NOS NAVEGADORES WEB

1. Interatividade e Dinamismo:

JavaScript permite que os desenvolvedores criem páginas web interativas, onde os usuários podem interagir com elementos da página, como botões e formulários, sem precisar recarregar a página.

2. Compatibilidade Universal:

JavaScript é suportado por todos os navegadores modernos, o que significa que os desenvolvedores podem escrever código que funcionará em uma ampla variedade de plataformas e dispositivos.

3. Frameworks e Bibliotecas:

A existência de uma vasta gama de frameworks e bibliotecas (como React, Angular e Vue.js) facilita o desenvolvimento de aplicações complexas, permitindo que os desenvolvedores reutilizem código e sigam boas práticas.

4. Desenvolvimento Full-Stack:

Com o advento do Node.js, os desenvolvedores podem usar JavaScript tanto no lado do cliente (navegador) quanto no lado do servidor, permitindo um desenvolvimento mais coeso e eficiente.



03. JAVASCRIPT COM NODE.JS

O QUE É NODE.JS?

- Node.js é uma plataforma de desenvolvimento baseada em JavaScript que permite criar aplicações como servidores e ferramentas de linha de comando.
- É um ambiente de execução de código aberto e funciona em diferentes sistemas operacionais, como Windows, macOS e Linux. Seu modelo de I/O não bloqueante permite operações assíncronas, resultando em alto desempenho para aplicações com múltiplas conexões simultâneas.
- O Node.js também possui o NPM (Node Package Manager), que oferece uma vasta biblioteca de módulos, facilitando o desenvolvimento. Essas características fazem do Node.js uma escolha popular para aplicações escaláveis e eficientes.



VUE.JS: FRAMEWORK JAVASCRIPT

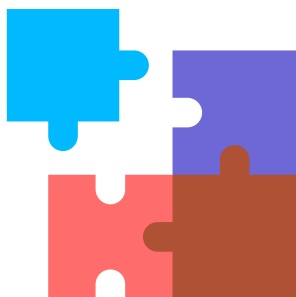
O QUE É O VUE.JS?

Criado por Evan You em 2014, O Vue.js é um framework JavaScript progressivo, amplamente utilizado para a criação de interfaces de forma componentizada semelhante ao React.

Ele se destaca pelo seu desempenho, curva de aprendizado baixa, flexibilidade e capacidade de ser integrado gradualmente em projetos existentes.

Principais Características do Vue.js

1. **Reatividade de Dados** - A reatividade do Vue funciona de tal forma que o desenvolvedor não necessita, manipular o DOM diretamente
2. **Componentização** - No Vue a componentização funciona de forma muito intuitiva com arquivos de extensão .vue onde neles temos tags como template, script e style.



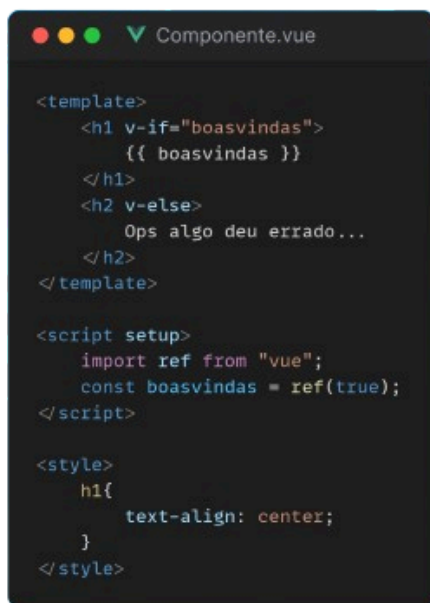
```
Componente.vue

<template>
  <h1>
    {{ boasvindas }}
  </h1>
</template>

<script setup>
  import ref from "vue";
  const boasvindas = ref("Seja bem vindo")
</script>

<style>
  h1{
    text-align: center;
  }
</style>
```

3. **Diretivas** - O Vue.js oferece diretivas como v-bind, v-for e v-if, que ajudam a manipular dinamicamente elementos HTML. Por exemplo, v-for é usado para renderizar listas de forma dinâmica.

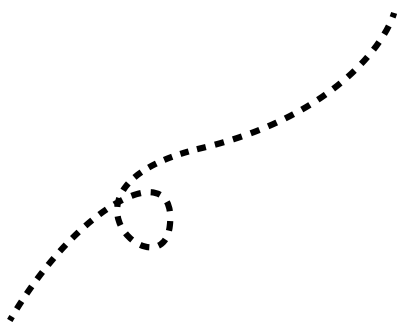


```
Componente.vue

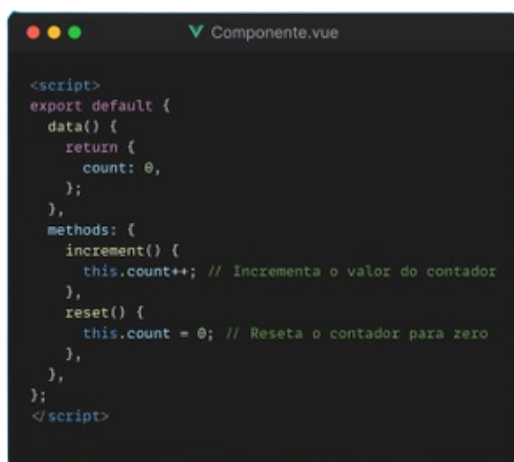
<template>
  <h1 v-if="boasvindas">
    {{ boasvindas }}
  </h1>
  <h2 v-else>
    Ops algo deu errado...
  </h2>
</template>

<script setup>
  import ref from "vue";
  const boasvindas = ref(true);
</script>

<style>
  h1{
    text-align: center;
  }
</style>
```



4. **OptionsAPI** - É uma forma que temos de realizar a Reatividade de dados no Vue quando estamos utilizando o Vue 3. Nele temos data, methods e computed. Onde conseguimos armazenar dados, manipular dados e utilizar dados de forma reativa assim evitando redundância.



```
Componente.vue

<script>
export default {
  data() {
    return {
      count: 0,
    };
  },
  methods: {
    increment() {
      this.count++; // Incrementa o valor do contador
    },
    reset() {
      this.count = 0; // Reseta o contador para zero
    },
  },
};
</script>
```

5. **CompositonAPI** - É a forma mais recente de realizar reatividade com o Vue. CompositonAPI surgiu no Vue 3, e realizou uma mudança considerável e facilitando ainda mais para quem está migrando de aplicações vanilla ou outros frameworks. Na composonAPI não temos a necessidade de utilizarmos um objeto e atribuímos data, computed e methods.



```
<!-- Componente.vue -->

<template>
  <h1 v-if="boasvindas">
    {{ boasvindas }}
  </h1>
  <h2 v-else>
    Ops algo deu errado...
  </h2>
</template>

<script setup>
  import ref from "vue";
  const boasvindas = ref(true);
</script>

<style>
  h1{
    text-align: center;
  }
</style>
```



Caso tenha se interessado pelo VueJS aqui estão algumas bibliotecas para você interagir e deixar sua aplicação VueJS Incrível.

Bibliotecas: Vue-Router, Pinia, PrimeVue, OhVuelcons e VueToastification

OBS: Lembre-se sempre de ler a documentação se houver dúvidas.

Vue Docs: <https://vuejs.org/>

VERSIONAMENTO COM GIT

INTRODUÇÃO:

O que é versionamento?

Versionamento é o processo de rastrear e gerenciar mudanças em arquivos ao longo do tempo. Ele **permite que desenvolvedores colaborem, testem novas ideias e revertam alterações quando necessário.**

Por que usar Git para controle de versão?

O Git é um sistema de controle que:

- Facilita o trabalho em equipe.
- Permite o rastreamento de alterações detalhadamente.
- É rápido, eficiente e amplamente adotado pela comunidade de desenvolvedores.

Atenção! Diferenças entre Git e GitHub

- **Git:** Ferramenta para controle de versão usada localmente ou em servidores.
- **GitHub:** Plataforma online para hospedar repositórios Git e facilitar a colaboração.

CONFIGURAÇÃO INICIAL

Como instalar o Git?

- No Windows: Baixe o instalador em git-scm.com.
- No macOS: Use o Homebrew:

```
brew install git
```

- No Linux:

```
sudo apt-get install git
```

Configuração do usuário

Configure suas informações para identificar suas alterações:

```
git config --global user.name "Seu Nome"  
git config --global user.email "seuemail@email.com"
```



COMANDOS BÁSICOS DO GIT

- Criar um repositório;

```
git init
```

- Adicionar arquivos ao GitHub;

```
git add .
```

- Salvar alterações(commit);

```
git commit -m "Mensagem do commit"
```

- Visualizar o status do repositório;

```
git status
```

- Histórico de alterações;

```
git log
```



TRABALHANDO COM REPOSITÓRIOS REMOTOS NO GITHUB

Vamos aprender a criar um repositório no GitHub:

1. Acesse o GitHub e crie um novo repositório.
2. Copie o URL do repositório.

Conectar um repositório local ao GitHub

```
git remote add origin  
https://github.com/seu-usuario/seu-  
repositorio.git  
  
git push -u origin main
```

- E como clonar um Repositório Existente?

O comando **git clone** é usado para copiar um repositório remoto para o seu computador. Ele cria uma cópia completa, incluindo todo o histórico de commits. Passos para clonar um repositório do GitHub:

1. Acesse o repositório no GitHub.
2. Clique no botão "Code" e copie o URL.
3. No terminal, execute o comando:

```
git clone <URL-do-repositório>
```



FLUXO DE TRABALHO COM BRANCHES

O que são branches e para que servem?

Branches permitem que você trabalhe em funcionalidades ou correções de forma isolada, sem interferir no código principal.

Criar e mudar de branch

```
git branch nova-branch  
git checkout nova-branch
```

Ou crie e mude ao mesmo tempo:

```
git checkout -b nova-branch
```

Mesclar branches (merge)

```
git checkout main  
git merge nova-branch
```



COLABORAÇÃO NO GITHUB

Fazer fork de um repositório:

O fork é uma cópia de um repositório existente, criada na sua conta GitHub. Ele é usado para contribuir com projetos sem modificar diretamente o repositório original.

Passos para fazer um fork:

1. Acesse o repositório no GitHub que você deseja modificar.
2. Clique no botão "Fork" no canto superior direito da página. (Isso criará uma cópia do repositório original no seu perfil do GitHub.)

Clonar o repositório forkeado:

Após criar o fork, você precisa clonar o repositório para o seu computador para começar a trabalhar.

1. No seu perfil, abra o repositório forkeado;
2. Clique em "Code" e copie o URL;
3. No terminal, execute:

```
git clone https://github.com/seu-  
usuario/repo-forkeado.git
```

Agora você pode modificar o código localmente!



COLABORAÇÃO NO GITHUB

Abrir um Pull Request:

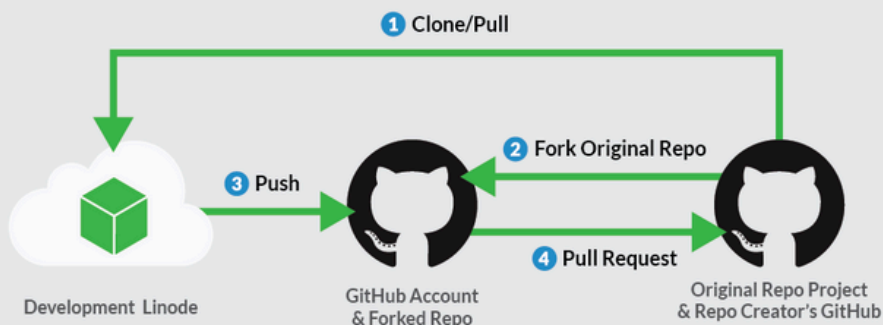
Um Pull Request (PR) é uma solicitação para que as alterações feitas no seu fork sejam revisadas e, possivelmente, integradas ao repositório original.

Passos para abrir um pull request:

1. Acesse o seu repositório forkeado no GitHub.
2. Clique no botão "Contribute" ou "Pull Request".
3. Escolha a branch do seu fork onde você fez as alterações e selecione a branch do repositório original para onde deseja enviar o PR (geralmente, a branch main ou develop).
4. Adicione um título e uma descrição clara para o PR, explicando as mudanças feitas.
5. Clique em "Create Pull Request".

Após abrir o Pull Request:

- O proprietário do repositório original revisará suas alterações.
- Caso aprovado, suas mudanças serão mescladas ao código principal.
- Se necessário, você pode receber feedback e ser solicitado a ajustar o código antes de aprovação.



DICAS PARA COLABORAR COM SUCESSO:

Colaborar em projetos no GitHub é uma oportunidade de aprender, compartilhar conhecimento e contribuir para o crescimento da comunidade de desenvolvedores.

Aqui estão algumas práticas essenciais para garantir uma colaboração eficiente:

1. Entenda o projeto antes de contribuir!

- Leia a documentação do projeto, como o arquivo README.md e as diretrizes de contribuição;

2. Mantenha seu Repositório atualizado

- Sincronize regularmente seu fork com o repositório original;

3. Faça alterações em Branches separadas

- Crie branches específicas para cada funcionalidade ou correção. Isso mantém o código organizado e facilita revisões.

4. Escreva mensagens de Commit claras

- Use mensagens descritivas para explicar o que foi alterado;

5. Comunique-se bem

- Seja claro e objetivo ao descrever seu Pull Request.
- Explique como suas alterações resolvem problemas ou melhoram o projeto.

6. Teste antes de enviar!

- Sempre teste suas alterações localmente para garantir que não introduziu erros.

05. ESLINT: GARANTINDO QUALIDADE NO CÓDIGO

Para que serve?



É possível executar um Linter no código para identificar e corrigir erros pequenos de programação ou práticas de códigos não convencionais que possam ter erros. Ele pode além da quantidade de linhas escritas, foca na legibilidade, simplicidade e eficiência do código.

```
1  async function buscarEDetalharNave() {
2    ⚠ try {
3      const resposta = await fetch("https://swapi.dev/api/starships/9/");
4      const nave = await resposta.json();
5
6      const detalhesNave = `
7        Nome: ${nave.name}
8        Modelo: ${nave.model}
9        Fabricante: ${nave.manufacturer}
10       `;
11
12      console.log("Detalhes da Nave:");
13      console.log(detalhesNave);
14
15      const tamanhoTripulacao = parseInt(nave.crew);
16      const tribulacaomaximo = 100;
17      if (tamanhoTripulacao > tribulacaomaximo) {
18        console.log("Esta é uma nave grande.");
19      } else {
20        console.log("Esta é uma nave pequena.");
21      }
22
23    } catch (erro) {
24      console.error("Erro ao buscar detalhes da nave:", erro);
25    }
26  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
99:26 error No magic number: 9 no-magic-numbers
99:29 error No magic number: 11 no-magic-numbers
102:25 error No magic number: 3 no-magic-numbers
118:50 error No magic number: 2 no-magic-numbers
118:53 error No magic number: 3 no-magic-numbers
119:68 error No magic number: 10 no-magic-numbers
```

✖ 109 problems (109 errors, 0 warnings)

Como utilizar ESLint no seu projeto?

npm i eslint -D

Vai fazer o download do eslint no seu projeto na dependência desenvolvimento.

npm run eslint --init

Configuração do eslint no projeto.

Algumas perguntas como

1. Como você gostaria de usar o ESLint?

```
@eslint/create-config: v1.4.0
? How would you like to use ESLint? -
  To check syntax only
  > To check syntax and find problems
```

Apenas para verificar a sintaxe

Se quer encontrar erros de sintaxe e problema

2. Que tipo de módulo você quer usar no seu projeto?

```
How would you like to use ESLint? · problems
What type of modules does your project use? ·
JavaScript modules (import/export)
CommonJS (require/exports)
None of these
```

JavaScript modules (import/export) versão mais atual

CommonJS (require/exports) versão antiga

Nenhum desses

3. Qual estrutura seu projeto usa?

```
@eslint/create-config: v1.4.0
✓ How would you like to use ESLint? · problems
✓ What type of modules does your project use? · esm
? Which framework does your project use? -
  > React
  Vue.js
  None of these
```

React

Vue.js

Nada disso

4. Seu projeto usa TypeScript?

```
@eslint/create-config: v1.4.0
✓ How would you like to use ESLint? · problems
✓ What type of modules does your project use? · esm
✓ Which framework does your project use? · none
? Does your project use TypeScript? _
  · No
  · Yes
```

Sim

Não

5. Onde seu código é executado?

```
✓ How would you like to use ESLint? · problems
✓ What type of modules does your project use? · esm
✓ Which framework does your project use? · none
✓ Does your project use TypeScript? · javascript
? Where does your code run? _ (Press <space> to select, <a> to toggle all, <i> to invert)
✓ Browser
✓ Node
```

Browser

Node

6. Você gostaria de instalar ESLint e suas dependências?

```
@eslint/create-config: v1.4.0
✓ How would you like to use ESLint? · problems
✓ What type of modules does your project use? · esm
✓ Which framework does your project use? · none
✓ Does your project use TypeScript? · javascript
✓ Where does your code run? · node
The config that you've selected requires the following dependencies:
eslint, globals, @eslint/js
? Would you like to install them now? · No / Yes
```

Sim

Não

7. Qual gerenciador de pacotes você quer usar?

```
✓ Would you like to install them now? · No / Yes
? Which package manager do you want to use? _
  · npm
  · yarn
  · pnpm
  · bun
```

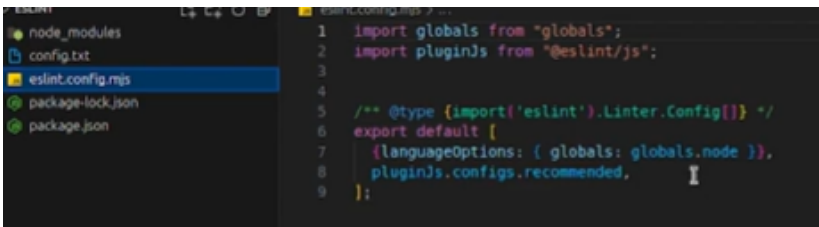
npm

yarn

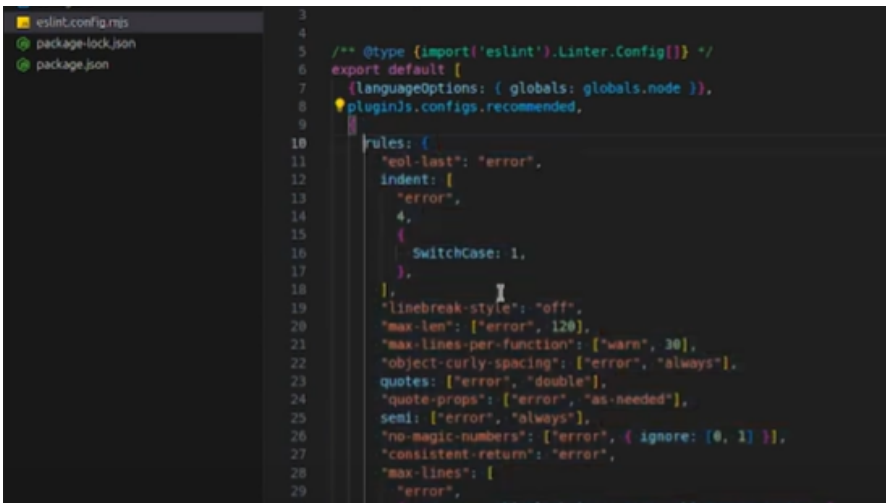
pnpm

bun

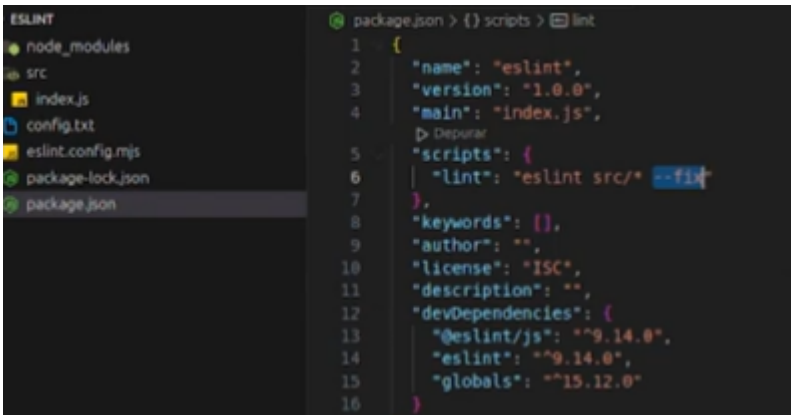
Com isso você configurar o ESLint no seu projeto



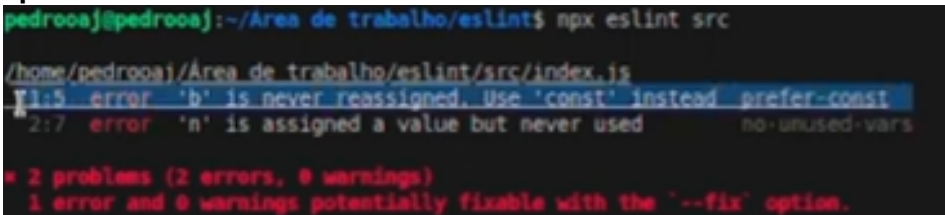
O próximo tem que colocar as regras:



Para executar o ESLint no seu projeto nomear na área scripts



Agora ele vai estar funcionando no seu projeto usado npx eslint src



06. VIRTUALIZANDO APLICAÇÕES: DOCKER

No mundo do desenvolvimento de software, a necessidade de criar ambientes consistentes, escaláveis e portáteis nunca foi tão importante. É nesse contexto que o Docker se destaca como uma das tecnologias mais revolucionárias da última década.

Docker é uma **plataforma de containerização** que permite empacotar uma aplicação e suas dependências em um contêiner leve, portátil e eficiente.

- É uma Plataforma de Código aberto para **criar, gerenciar e executar aplicações em containers**.





E os containers?

Na computação, um container é uma unidade isolada e independente que empacota:

- Código
- Bibliotecas
- Dependências
- Variáveis de ambiente

E a containerização?

A containerização é o termo dado a prática de utilizar os containers para isolar e empacotar as aplicações.

Por que usar Docker no projeto?

- **Isolamento: Cada projeto no seu "próprio espaço"**

Imagine que você tem dois projetos diferentes, e cada um precisa de uma versão diferente de uma ferramenta. No seu computador, seria confuso instalar essas versões ao mesmo tempo. O Docker resolve isso criando "caixinhas" separadas (chamadas de contêineres) onde cada projeto tem as ferramentas que precisa, sem interferir no outro.

- **Portabilidade: Funciona em qualquer lugar**

Já aconteceu de algo funcionar no seu computador, mas dar erro quando você tenta rodar em outro? Com Docker, você coloca tudo que o seu projeto precisa dentro de uma "caixinha" que pode ser levada para qualquer lugar — outro computador, um servidor ou até a nuvem — e ela vai funcionar exatamente igual.

- **Automatização: Tudo pronto com um comando**

Com o Docker, você não precisa mais perder tempo instalando coisas manualmente ou configurando tudo do zero em cada máquina. Você escreve um "manual" (um arquivo chamado Dockerfile) dizendo ao Docker o que fazer, e ele monta tudo sozinho para você. Assim, é muito mais rápido colocar o projeto para funcionar!



Conclusão e Reflexão

Este e-book foi criado para consolidar os aprendizados da disciplina de Ambientes e Desenvolvimento de Software, abordando ferramentas essenciais como JavaScript, Vue.js, Node.js, Git, GitHub e Docker. Cada capítulo une teoria e prática, com exemplos aplicáveis ao dia a dia de desenvolvimento, oferecendo uma base sólida para quem deseja aprofundar conhecimentos nessas tecnologias.

Durante a criação deste material, enfrentamos diversos desafios, como a compreensão de conceitos fundamentais em JavaScript, a configuração de ferramentas como Docker e ESLint, e a organização no uso de Git e GitHub para gerenciar projetos colaborativos. Além disso, a criação de aplicações modulares com Vue.js e Node.js exigiu esforço para integrar diferentes tecnologias de forma eficiente e escalável.

Apesar dessas dificuldades, cada obstáculo contribuiu para nosso desenvolvimento, fortalecendo habilidades técnicas e comportamentais, como resiliência, paciência e trabalho em equipe. Os aprendizados foram diversos: desde a importância de manter um código limpo e padronizado com ESLint até a versatilidade do Docker para criar ambientes de desenvolvimento consistentes e implantar aplicações de forma ágil. Também aprendemos como Git e GitHub facilitam o gerenciamento de projetos e como frameworks como Vue.js simplificam o desenvolvimento de interfaces interativas.

Encerramos este projeto com a certeza de que ele representa apenas o início de uma longa jornada. O desenvolvimento de software é um campo em constante evolução, e os conhecimentos adquiridos aqui são uma base para enfrentar novos desafios.

eBook Ambientes de Dev. de Software

GUIA PRÁTICO COM JAVASCRIPT, VUE E NODE.JS

Feito por:

Pedro Antônio Mendes Lemos

Teresa Cristina Luz Costa Campos

João Elfredo Gonsalves da Costa

Emanuel Sales Marinho Rocha

Caua de Noroes Milfont Queiroz

Ismael Marinho de Oliveira

Universidade de Fortaleza
2024.2